

# Analytic Proof Certificates for Forge

Canonical differential ladders, auxiliary factors,  
and explicit eventual bounds

A technical article with executable exact-arithmetic prototypes

Prepared for Vladimir Reshetnikov

September 15, 2026

**The proposed extension.** Turn inequalities involving exponentials, logarithms, and polynomial factors into small certificates whose analytic content is a single integrating-factor lemma. Add a total eventual-sign procedure that constructs explicit rational cutoffs rather than merely asserting domination.

## Principal results

A nondecreasing factor order dominates every permutation of the same differential factors. Extra factors form a monotone hierarchy of stronger positivity certificates. An interior-zero obstruction identifies when that hierarchy cannot help. Explicit tail bounds complement the ray certificates.

## Evidence boundary

284 separately bound certificate receipts replay under standard-library-only Python; 18 unit-test methods pass. The factor-order ablation checks 14,409 permutations on 200 inputs. Lean bridge and replay sources are supplied as *uncompiled candidates*; this archive does not contain an implemented `forge` tactic or a comparison against `grind`. Audited Forge snapshot: `c98e47c5f804e92880fc1d0e378c1b95832b685c`.

## Abstract

The existing Forge design already contains extensive polynomial arithmetic, structural induction, witness synthesis, and finite certificates for unbounded discrete behaviour. This article develops a complementary analytic capability: exact sign certificates for rational exponential polynomials and for source expressions reducible to them by proved changes of variable.

The central object is a differential ladder  $q_{j+1} = (D - \rho_j)q_j$  with a zero terminal function and nonnegative initial values. Its soundness follows from an integrating factor. A swapping argument eliminates factorial search over factor order: ascending order succeeds whenever any ordering of the same multiset can succeed. Additional factors enlarge the accepted cone, but a strict-positivity theorem shows that nonzero ladders cannot certify an interior zero. This gives a mathematical routing criterion rather than another budget heuristic. A second worker computes a certified eventual sign and explicit cutoff for every nonzero member of the exact fragment.

The article supplies proofs, sparse representations, certificate formats, source-translation obligations, an implementation sequence, reproducible experiments, adversarial checks, and candidate Lean proof emission. It separates new algorithms from existing libraries to reuse, and mathematical guarantees from the narrower properties actually established by the accompanying code.

## Contents

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>The gap this extension addresses</b>                         | <b>1</b> |
| 1.1      | A delta against the repository, not another Forge introduction  | 1        |
| 1.2      | What Leant and Djex contribute to this design                   | 1        |
| 1.3      | Mathematical provenance and the intended claim                  | 2        |
| <b>2</b> | <b>The exact fragment and its representation</b>                | <b>2</b> |
| 2.1      | Rational exponential polynomials                                | 2        |
| 2.2      | Finite-dimensional differential structure                       | 3        |
| 2.3      | Source-language boundaries                                      | 3        |
| 2.4      | Why the origin is an especially good anchor                     | 3        |
| <b>3</b> | <b>Differential ladders as positivity certificates</b>          | <b>4</b> |
| 3.1      | The analytic lemma                                              | 4        |
| 3.2      | The certificate and its soundness                               | 4        |
| 3.3      | A certificate which tolerates arbitrarily high boundary contact | 5        |
| 3.4      | The linear-algebra interpretation                               | 5        |
| 3.5      | A minimal checker loop                                          | 6        |
| <b>4</b> | <b>Canonical factor order and an enrichment hierarchy</b>       | <b>6</b> |
| 4.1      | Factorial search is unnecessary                                 | 6        |
| 4.2      | Adding factors without changing the source function             | 7        |
| 4.3      | A bounded but useful search strategy                            | 8        |
| 4.4      | An obstruction that no larger auxiliary budget repairs          | 9        |
| 4.5      | Towards better enrichment, without changing the checker         | 9        |
| <b>5</b> | <b>A total eventual-sign worker with explicit witnesses</b>     | <b>9</b> |
| 5.1      | Why eventual goals deserve their own certificate                | 9        |
| 5.2      | An explicit domination theorem                                  | 10       |
| 5.3      | The search and the checker                                      | 11       |
| 5.4      | Why sufficiently large native anchors work mathematically       | 11       |
| 5.5      | Quantitative asymptotics available from the same proof          | 11       |
| 5.6      | Combining a tail with a bounded-domain proof                    | 12       |

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>6</b> | <b>Exact initial values and bounded-domain coverage</b>                 | <b>12</b> |
| 6.1      | Rational bounds for an exponential . . . . .                            | 12        |
| 6.2      | Normalising an entire point evaluation . . . . .                        | 13        |
| 6.3      | A reference interval cover, not a new interval tactic . . . . .         | 13        |
| 6.4      | Why subdivision cannot replace the ladder . . . . .                     | 14        |
| 6.5      | Negative points are useful, but have a different type . . . . .         | 14        |
| <b>7</b> | <b>From source expressions to Lean proofs</b>                           | <b>14</b> |
| 7.1      | A small semantic bridge rather than an unverified translation . . . . . | 14        |
| 7.2      | Logarithmic charts expand the useful source language . . . . .          | 15        |
| 7.3      | Composing the worker with existing algebra . . . . .                    | 16        |
| 7.4      | Concrete libraries to reuse . . . . .                                   | 16        |
| 7.5      | Candidate proof emission in the archive . . . . .                       | 17        |
| 7.6      | Why a reflected checker is the second step, not the first . . . . .     | 18        |
| <b>8</b> | <b>Executed experiments and what they establish</b>                     | <b>18</b> |
| 8.1      | Environment, inputs, and receipts . . . . .                             | 18        |
| 8.2      | An ablation with an exhaustive finite oracle . . . . .                  | 19        |
| 8.3      | Independent calculations and mutation tests . . . . .                   | 19        |
| 8.4      | Unknown results retained deliberately . . . . .                         | 20        |
| 8.5      | Replay separation and its limits . . . . .                              | 20        |
| <b>9</b> | <b>An implementation sequence for Forge</b>                             | <b>20</b> |
| 9.1      | The first useful end-to-end slice . . . . .                             | 20        |
| 9.2      | Milestones with exact acceptance conditions . . . . .                   | 21        |
| 9.3      | Routing should exploit the theorems . . . . .                           | 22        |
| 9.4      | Proof-cost controls . . . . .                                           | 22        |
| 9.5      | A benchmark plan that would support real capability claims . . . . .    | 22        |
| 9.6      | What would constitute a significant gain . . . . .                      | 23        |
| <b>A</b> | <b>Certificate formats and API contracts</b>                            | <b>23</b> |
| A.1      | Canonical source objects . . . . .                                      | 23        |
| A.2      | Ladder receipts . . . . .                                               | 24        |
| A.3      | Tail receipts . . . . .                                                 | 24        |
| A.4      | The accepted source request types . . . . .                             | 25        |
| <b>B</b> | <b>Reproduction and archive map</b>                                     | <b>25</b> |
| B.1      | Run the executable evidence . . . . .                                   | 25        |
| B.2      | Build the article and inspect the candidates . . . . .                  | 26        |
| B.3      | Files worth opening first . . . . .                                     | 26        |

# 1 The gap this extension addresses

## 1.1 A delta against the repository, not another Forge introduction

The audited Forge snapshot is `c98e47c5f804e92880fc1d0e378c1b95832b685c`. Its merged article already treats polynomial cones, exact quadratic decomposition, Bernstein and univariate certificates, structural generalisation, witness construction, discrete observable-space and ideal closure, finite-algebra covers, telescoping, and operator transport. Its status ledger distinguishes executed Python prototypes, emitted Lean candidates, and unimplemented runtime components. Those mechanisms are prerequisites here, not contributions to be reintroduced under new names [1, 2, 3, 4].

The narrow addition is *analytic sign information*. For example, replacing  $e^x$  by a fresh positive real variable discards precisely the relationship to  $x$  needed to establish

$$e^x \geq 1 + x, \quad e^x \geq \sum_{k=0}^n \frac{x^k}{k!} \quad (x \geq 0), \quad \exists X \forall x \geq X : e^x > 1000x^5.$$

Polynomial reasoning alone does not recover that relationship from an opaque atom. It needs a proved analytic consequence, a derivative identity, or a change of variable which exposes the missing structure.

This article does not assert that existing Lean tactics fail on these displayed examples. Several have short library proofs. Their purpose is to exhibit the input language and proof certificates, not to construct a weak baseline. The proposed capability is a systematic worker for expanded, unfamiliar combinations of such terms, including automatic cutoffs and useful routing diagnostics.

| Concern             | Already present in Forge                  | Addition here                                                                      |
|---------------------|-------------------------------------------|------------------------------------------------------------------------------------|
| Positivity          | Exact polynomial certificates             | Derivative-linked exponential-polynomial certificates                              |
| Unbounded reasoning | Discrete invariant and observable closure | Positivity on a continuous ray; explicit eventual sign                             |
| Search structure    | Candidate generation and closure          | A canonical factor order, with a proof eliminating order search                    |
| Failure handling    | Bounded failure is unknown                | A proved interior-zero obstruction, and a precise alternate route                  |
| Lean integration    | General proof and source contracts        | A concrete analytic step theorem, sparse checker obligations, and a replay emitter |

The audit read the merged repository’s overview, status, nonlinear arithmetic, closure, and conclusion material, and examined neighbouring indexing diagnostics. It was not a line-by-line review of every historical proposal. Consequently the claim is a substantive delta against those documented capabilities, not a claim that no related word or idea appears anywhere in the repository.

## 1.2 What Leant and Djex contribute to this design

The inspected neighbouring snapshots are Leant `6bf05ad78c467989e68290f2d08bbbed40802d485` and Djex `e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5`. Their current indexing diagnostics make a useful

distinction: synthesising the right components separately does not establish that the original bounded query has been solved. The original integer-indexing acceptance remains absent in those diagnostics [5, 6].

There is an exact analytic analogue. A certificate for  $f(x) > 0$  when  $x \geq 32$  does not close a request for  $f(x) \geq 0$  when  $x \geq 0$ . A proof about the transformed exponential polynomial does not close a source theorem until the logarithmic chart and its domain have been proved. A generated nonnegativity theorem does not close a strict-positivity request. Accordingly the prototype has a separate `verify_goal` layer, not merely a predicate saying that a receipt is internally consistent. These particular distinctions are enforced by negative tests, rather than being left as aspirations in a general trust model.

### 1.3 Mathematical provenance and the intended claim

The ingredients are classical: constant-coefficient differential equations, integrating factors, real exponential-polynomial function spaces, and exact Taylor bounds. Extended Chebyshev systems provide a broader setting for related ordered bases [12]; differential-invariant proof systems address a much larger class of continuous dynamics [11]. Nothing here claims invention of differential induction or of positive bases.

The contribution relative to Forge is the particular algorithmic package: a small accepted language, an ordering theorem that removes an apparent search dimension, an exact enrichment hierarchy, a useful impossibility criterion for that hierarchy, a total explicit tail constructor, and detailed lowering to ordinary Lean theorems. The proofs are included so that these claims need not rest on an analogy with a larger literature.

**Implementation status in one place.** The sparse algebra, ladder search, auxiliary-factor search, rational enclosures, tail search, reference interval cover, source-goal binding, mutation checks, and replay experiments were executed in Python. Four Lean replay files and a candidate analytic bridge were generated, but not compiled. The semantic Lean reifier, reflected checker, integrated `forge` worker, and Lean benchmark suite remain implementation work. Later sections do not silently promote one status to another.

## 2 The exact fragment and its representation

### 2.1 Rational exponential polynomials

**Definition 2.1.** Let  $\mathcal{E}_{\mathbb{Q}}$  be the algebra of functions of the form

$$f(x) = \sum_{\lambda \in \Lambda} P_{\lambda}(x) e^{\lambda x}, \quad \Lambda \subset \mathbb{Q} \text{ finite}, \quad P_{\lambda} \in \mathbb{Q}[x]. \quad (1)$$

The stored representation combines equal rates, combines equal powers, removes zero coefficients, and removes zero polynomial blocks.

A dictionary entry  $(\lambda, k) \mapsto c$  denotes  $c x^k e^{\lambda x}$ . All three components are exact:  $\lambda, c$  are reduced rationals and  $k$  is a nonnegative integer. The zero map denotes the zero function. Polynomial terms are simply the block with rate zero. Negative rates are first-class; rational rates such as  $1/2$  require no special case.

Addition merges dictionaries. Multiplication uses

$$(c x^k e^{\lambda x})(d x^j e^{\mu x}) = c d x^{k+j} e^{(\lambda+\mu)x}.$$

Reflection, needed for the other half-line, replaces  $(\lambda, k, c)$  by  $(-\lambda, k, (-1)^k c)$ . Differentiation and a shifted derivative are

$$D(P_\lambda e^{\lambda x}) = (P'_\lambda + \lambda P_\lambda) e^{\lambda x}, \quad (2)$$

$$(D - \rho)(P_\lambda e^{\lambda x}) = (P'_\lambda + (\lambda - \rho)P_\lambda) e^{\lambda x}. \quad (3)$$

No transcendental arithmetic is required to perform any of these operations.

## 2.2 Finite-dimensional differential structure

For a nonzero canonical input define

$$d_\lambda = \deg P_\lambda, \quad N = \sum_{\lambda \in \Lambda} (d_\lambda + 1), \quad A_f(D) = \prod_{\lambda \in \Lambda} (D - \lambda)^{d_\lambda + 1}.$$

The function belongs to the  $N$ -dimensional space spanned by  $x^k e^{\lambda x}$  for  $0 \leq k \leq d_\lambda$ . The operator  $A_f(D)$  annihilates it. Indeed,  $(D - \lambda)$  differentiates the polynomial in the corresponding block, and all constant-coefficient factors commute. Repeating it  $d_\lambda + 1$  times kills that block; the other factors do not raise its degree.

This is not the repository's discrete zero-observation closure in new notation. The finite differential space is used here to propagate an *order inequality* through positive integrating factors on a continuous domain. A finite list of derivative identities alone would not establish that inequality.

The representation permits holes in a polynomial's coefficients. The dimension bound nevertheless uses its largest degree, not the number of nonzero terms: differentiation can fill those holes. A sparse implementation may store only nonzero entries, but must not confuse sparse support size with the length of an annihilator.

## 2.3 Source-language boundaries

A first Lean reifier should recognise real-valued expressions generated by rational constants, one distinguished real local variable, addition, subtraction, multiplication, natural powers, and  $\text{Real.exp}(rx)$  with rational  $r$ . It should produce both the canonical object and a proof that evaluation agrees with the source expression. An expression hash is useful for caching but is not that proof.

Several natural-looking extensions require additional work. A shifted argument  $e^{rx+b}$  factors as  $e^b e^{rx}$ , but  $e^b$  need not be rational; it is outside (1) unless a positive common factor can be removed or the coefficient domain is deliberately enlarged. Likewise  $2^x$  has rate  $\log 2$ , which is not a rational rate. The analytic ladder theorem allows arbitrary real factors, but the delivered search and canonical arithmetic are rational. The prototype does not pretend that a rational approximation to  $\log 2$  gives an exact source translation.

Expressions such as  $e^{x^2}$ ,  $\sin x$ , nested exponentials, and general multivariate exponential sums are not accepted by this reifier. Some can be reduced to the fragment by a proved substitution, but there is no implicit fallback which treats an unsupported operation as an unrelated formal symbol.

## 2.4 Why the origin is an especially good anchor

At  $x = 0$ , every exponential is one and all positive powers of  $x$  vanish:

$$f(0) = \sum_{\lambda} P_\lambda(0) \in \mathbb{Q}.$$

Every member of a derivative ladder still has the same form. Thus every initial sign at the origin is an exact rational comparison, including equality to zero. No precision loop is needed to certify a multiple boundary zero.

At a rational anchor  $a \neq 0$ , the coefficients and operators remain rational, but initial values are finite sums of rational multiples of  $e^{\lambda a}$ . They require proved enclosures, or a separate symbolic cancellation argument. The implementation evaluates  $P_\lambda(a)$  exactly before approximating any exponential. Canonical cancellation at this stage is essential: a genuinely zero value must not be rediscovered by trying to squeeze an interval of uncertain sign to an impossible finite width.

### 3 Differential ladders as positivity certificates

#### 3.1 The analytic lemma

**Lemma 3.1** (One integrating-factor step). *Let  $f, g$  be real functions on  $[a, \infty)$ , with  $f$  continuously differentiable and  $g$  continuous. Suppose*

$$f'(x) = \rho f(x) + g(x), \quad f(a) \geq 0, \quad g(x) \geq 0 \quad (x \geq a),$$

where  $\rho \in \mathbb{R}$ . Then  $f(x) \geq 0$  on  $[a, \infty)$ .

*Proof.* For  $w(x) = e^{-\rho x} f(x)$ , the product and chain rules give  $w'(x) = e^{-\rho x} g(x) \geq 0$ . Hence  $w(x) \geq w(a) \geq 0$  for  $x \geq a$ . Multiplication by the positive number  $e^{\rho x}$  gives the conclusion. Equivalently, the exact identity

$$f(x) = e^{\rho(x-a)} f(a) + \int_a^x e^{\rho(x-t)} g(t) dt \quad (4)$$

expresses both terms as nonnegative. □

The derivative proof is preferable for a first Lean bridge. It needs only explicit derivative witnesses, continuity, and monotonicity on a convex ray; it does not require construction of an integral expression or a separate integrability proof for every emitted step. Mathlib exposes these ingredients through `HasDerivAt`, exponential derivative lemmas, and the `monotoneOn_of_hasDerivWithinAt_nonneg` theorem [7, 8].

#### 3.2 The certificate and its soundness

**Definition 3.2.** A differential-ladder certificate for  $f$  at  $a$  consists of real numbers  $\rho_0, \dots, \rho_{n-1}$  and functions  $q_0, \dots, q_n$  such that

$$q_0 = f, \quad q_{j+1} = q'_j - \rho_j q_j, \quad q_n = 0, \quad q_j(a) \geq 0 \quad (0 \leq j < n). \quad (5)$$

In the executable fragment, all  $q_j$  are canonical rational exponential polynomials and all  $\rho_j, a$  are rational.

**Theorem 3.3** (Ladder soundness). *A certificate satisfying (5) proves  $f(x) \geq 0$  for every  $x \geq a$ .*

*Proof.* The terminal function is nonnegative. Descend from  $q_n$  to  $q_0$  using Lemma 3.1, with  $f = q_j$  and  $g = q_{j+1}$  at each step. Every required smoothness condition holds for exponential polynomials. □

The checker does not need to know why the factors were chosen or whether they are minimal. In particular, it need not trust a spectral calculation. It checks the derivative identities, the initial inequalities, and the zero terminal object directly. An unnecessarily long but correct ladder is still a proof certificate.

**Theorem 3.4** (Strictness and zero obstruction). *Suppose a ladder is accepted. If some  $q_j(a) > 0$ , then  $f(x) > 0$  for every  $x > a$ . If  $q_0(a) > 0$ , positivity holds on the closed ray. If every  $q_j(a) = 0$ , then  $f$  is zero throughout the ray. Consequently a nonzero function represented by an accepted ladder has no zero in  $(a, \infty)$ .*

*Proof.* If  $q_j(a) > 0$ , equation (4) shows that  $q_j(x) > 0$  for  $x \geq a$ . For any  $x > a$ , its contribution to the next integral down is strictly positive. Repeating the argument gives  $q_0(x) > 0$ . When  $q_0(a) > 0$  the endpoint is positive too. If every initial value is zero, descend from  $q_n = 0$  in the same identity to obtain  $q_{n-1} = \dots = q_0 = 0$ .  $\square$

This theorem provides more than a strictness flag. It proves that there are true nonnegativity goals for which *no amount of ladder search* can work. Section 4 uses that fact to choose another worker.

### 3.3 A certificate which tolerates arbitrarily high boundary contact

Let

$$R_m(x) = e^x - \sum_{k=0}^m \frac{x^k}{k!}, \quad m \geq 0.$$

Take  $m + 1$  zero factors followed by one factor equal to one. Successive ordinary derivatives reduce the polynomial degree; after  $m + 1$  steps the function is  $e^x$ , and the final  $(D - 1)$  kills it. The initial values are

$$\underbrace{0, \dots, 0}_{m+1 \text{ values}}, 1.$$

Thus  $R_m(x) \geq 0$  for  $x \geq 0$  and  $R_m(x) > 0$  for  $x > 0$ . The certificate grows linearly in  $m$  in its number of steps. Its correctness does not become a numerical conditioning problem as the contact order increases.

For  $m = 4$  the exact ladder is

$$(0, 0, 0, 0, 0, 1), \quad \text{initial vector } (0, 0, 0, 0, 0, 1).$$

The delivered experiments exercise  $m = 0, \dots, 24$ . A strict closed-ray request is deliberately rejected because  $R_m(0) = 0$ ; an open-ray request is accepted.

### 3.4 The linear-algebra interpretation

Fix a multiset of  $n$  real factors and the space  $V$  of solutions of the associated homogeneous constant-coefficient equation. The map

$$f \mapsto (f(a), (D - \rho_0)f(a), \dots, (D - \rho_{n-2}) \cdots (D - \rho_0)f(a))$$

is a coordinate system on  $V$ . Its coordinates are triangular combinations of ordinary derivatives at  $a$ , with diagonal coefficients one. Equivalently, the first-order triangular system in (5) reconstructs a unique function from those initial data.

Nonnegative coordinates define a simplicial cone in  $V$ . Its basis functions can be constructed backwards from unit initial vectors using (4); they are positive on the open ray. Therefore ladder search is not an unstructured

search through arbitrary inequalities. It is membership testing in a particular positive cone, with a particularly simple change of coordinates.

This also explains both strengths and weaknesses. The cone is adapted to boundary zeros, since early coordinates may vanish exactly. But none of its nonzero elements can have an interior zero, because every nonzero nonnegative combination of its basis functions is strictly positive there. Adding more coordinates can enlarge the accepted cone without eliminating that obstruction.

### 3.5 A minimal checker loop

```
checkLadder(source, anchor, factors, receipt):
    q := source
    strictSeen := false
    for each factor rho and corresponding receipt step:
        require step.polynomial == q
        [lo, hi] := certifiedPointBounds(q, anchor)
        require step.bounds == [lo, hi] and lo >= 0
        strictSeen := strictSeen or (lo > 0)
        q := independentlyComputedShiftedDerivative(q, rho)
    require q == 0
    return NonnegativeRay(anchor, strictSeen)
```

The independently computed derivative in the prototype uses a coefficient stencil rather than the producer's termwise differentiation loop:

$$\tilde{c}_{\lambda,k} = (\lambda - \rho)c_{\lambda,k} + (k + 1)c_{\lambda,k+1}. \quad (6)$$

This is algorithmic diversity, not a formal verification of either Python implementation. The separately tested initial-jet oracle uses a third calculation, described in Section 8.

A production receipt need not retain every intermediate polynomial: the checker can reconstruct them from the source and factors. The reference format keeps them for diagnosis, mutation tests, and readable emitted proofs. Omitting those fields is a later certificate-size optimisation, not a change in the theorem.

## 4 Canonical factor order and an enrichment hierarchy

### 4.1 Factorial search is unnecessary

The annihilator of an exponential polynomial supplies a multiset of factors, but initially seems to leave their order to search. The initial coordinates change with the order, so arbitrary ordering can indeed fail. Fortunately, there is a best order in the relevant sense.

**Theorem 4.1** (Ascending-order dominance). *Fix a function, an anchor, and a multiset of constant real differential factors whose product annihilates the function. If any ordering of these factors gives nonnegative ladder initial values, then their nondecreasing ordering does too.*

*Proof.* Consider an adjacent inversion  $\alpha > \beta$  after a common prefix has produced  $q$ . In the original order the intermediate function is  $u = (D - \alpha)q$ . In the swapped order it is

$$\tilde{u} = (D - \beta)q = u + (\alpha - \beta)q.$$

The original successful ladder gives  $q(a) \geq 0$  and  $u(a) \geq 0$ , hence  $\tilde{u}(a) \geq 0$ . After both factors the functions are exactly equal, since  $(D - \alpha)$  and  $(D - \beta)$  commute. Every earlier and later initial value is unchanged, as is the terminal zero. Each adjacent inversion can therefore be removed without losing acceptance. Bubble sorting finishes.  $\square$

**Corollary 4.2.** *For the native multiset containing each rate  $\lambda$  exactly  $d_\lambda + 1$  times, checking one ascending ladder decides whether any permutation of that multiset admits a ladder certificate, provided the initial signs are determined exactly.*

The qualification is important. This is completeness for a *certificate class*, not for positivity of exponential polynomials. At the origin, the prototype's rational signs are exact, so it implements precisely that fixed-multiset test. At other anchors, its fixed-precision enclosures can fail to establish a true nonnegative initial value. The ordering theorem remains true, but bounded numerical sign discovery is an additional source of UNKNOWN.

With  $N$  native factors and at most  $S \leq N$  coefficients in the corresponding dense block representation, a straightforward implementation uses  $O(NS)$  rational arithmetic operations. Sorting rates costs  $O(|\Lambda| \log |\Lambda|)$  comparisons; repeated rates need not actually be sorted one copy at a time. Numerator and denominator growth must be accounted for separately. This is not a bit-complexity or timing claim.

The implementation should therefore never offer a large permutation budget for this lane. A failed ascending test is a reason to enrich the language, move the anchor, split the domain, factor the target, or route elsewhere. Reordering the same factors cannot reveal a successful certificate that ascending order missed at exact signs.

## 4.2 Adding factors without changing the source function

A fixed multiset can be too weak. Consider

$$f_\varepsilon(x) = e^{2x} - 4e^x + 4 + \varepsilon, \quad \varepsilon > 0. \quad (7)$$

The native factors are  $(0, 1, 2)$ . The first step tests  $f_\varepsilon(0) = 1 + \varepsilon > 0$ , but its successor has value  $f'_\varepsilon(0) = -2$ . Thus the native ladder fails, regardless of factor order by Theorem 4.1. Nevertheless  $f_\varepsilon = (e^x - 2)^2 + \varepsilon$  is strictly positive.

If  $A(D)f = 0$ , then  $(D - \beta)A(D)f = 0$  for every real  $\beta$ . We can therefore add factors without changing the input function. They change the coordinates used to establish positivity, not the theorem to be proved.

**Proposition 4.3** (Monotonicity under multiset extension). *If a function has an accepted ascending ladder for a multiset  $M$ , then it has an accepted ascending ladder for every finite multiset containing  $M$ .*

*Proof.* Append the additional factors after the existing terminal zero. Every new intermediate function and initial value is zero, so the extended unsorted ladder is accepted. Apply Theorem 4.1 to sort the full multiset.  $\square$

The enlarged cones can be strictly larger. For  $f_1 = e^{2x} - 4e^x + 5$ , four additional factors equal to  $-4$  give

$$(-4, -4, -4, -4, 0, 1, 2),$$

with initial vector

$$(2, 6, 16, 36, 76, 92, 2592).$$

Every entry is positive and the final function is zero. This is a completely rational certificate of strict positivity on  $[0, \infty)$ .

The prototype calls these *ghost factors*; the article generally calls them auxiliary factors. They are not additional program variables or new continuous state variables. In particular, the term is not a claim that this construction implements the differential-ghost rule of a general dynamic logic [11].

### 4.3 A bounded but useful search strategy

For a native minimum rate  $\lambda_{\min}$ , try

$$\beta \in \{\lambda_{\min} - 1, \lambda_{\min} - 2, \lambda_{\min} - 4, \dots, \lambda_{\min} - 64\}.$$

For each  $\beta$ , append repeated copies *before* the native ascending suffix. Because  $\beta$  is below the native spectrum, the combined order is already ascending. At each depth compute the next prefix function once and try the native suffix on it. If the current prefix initial value is negative, stop extending that particular prefix: every longer ladder contains the same failed test.

```
enrichedLadder(f, maxExtra):
  if nativeLadder(f) succeeds: return it
  native := ascendingNativeFactors(f)
  for beta in selectedRatesBelow(min(native)):
    q := f
    prefix := []
    repeat at most maxExtra times:
      if q(0) < 0: break
      prefix.append(beta)
      q := (D - beta) q
      if nativeLadder(q) succeeds:
        return checkLadder(f, 0, prefix ++ native)
  return Unknown
```

The experiment used at most 96 auxiliary factors per selected rate. Its results for (7) are:

| $\varepsilon$ | Native result | $\beta$ | Extra factors | Total factors |
|---------------|---------------|---------|---------------|---------------|
| 1             | unknown       | -4      | 4             | 7             |
| 1/2           | unknown       | -8      | 7             | 10            |
| 1/4           | unknown       | -16     | 13            | 16            |
| 1/16          | unknown       | -64     | 46            | 49            |

These rows demonstrate a strict increase in the expressive power of the ladder language. They are *not* a superiority result over Forge’s whole arithmetic portfolio: an algebraic worker can establish the displayed square-plus-constant identity immediately after introducing  $u = e^x$ . This family deliberately has a known proof, making it a clean control for the enrichment mechanism.

The present search is not claimed complete for strictly positive functions on a ray. The monotonicity proposition does not by itself show that the union of all auxiliary-factor cones exhausts the strictly positive cone. Establishing such a theorem, under explicit spectral and endpoint hypotheses, would be a separate mathematical result. It must not be inferred from four successful rows.

#### 4.4 An obstruction that no larger auxiliary budget repairs

Set  $\varepsilon = 0$ . The function  $(e^x - 2)^2$  is nonnegative on  $[0, \infty)$ , is not identically zero, and vanishes at  $x = \log 2 > 0$ . By Theorem 3.4, no finite ladder of the accepted form can establish its nonnegativity on that entire ray, even with arbitrary real auxiliary factors.

This is a structural failure, not evidence that the theorem is false. The correct route is to preserve or discover the square factor and use algebraic positivity. More generally, if the target factors as  $h(x)^2 r(x)$  and the algebraic identity is proved, send  $r$  to the analytic worker and keep the square in the algebraic layer. The zeros of  $h$  need not be located to prove nonnegativity. To prove strict positivity, however, the source goal must also supply or establish  $h \neq 0$  on the requested domain.

Another route is domain splitting at a known zero, but it needs a certified split point and new initial-value arguments on each piece. A numerical root approximation cannot be used as an exact domain boundary. Factoring is therefore often substantially cheaper than asking a transcendental root isolator to solve a problem that a square identity already settles.

#### 4.5 Towards better enrichment, without changing the checker

The simple search exposes several concrete optimisations. Cache the prefix polynomials so that increasing the number of copies does not recompute them. Evaluate native-suffix initial values directly from jets before constructing all suffix polynomials. Reject a proposed factor family using its earliest negative coordinate. Use the coefficient norm and the minimum observed positive coordinate to rank alternative  $\beta$  values, but do not make that heuristic part of acceptance.

A more general search can insert several distinct auxiliary rates. Theorem 4.1 still removes their ordering dimension. What remains is a search over multisets and their multiplicities. An LP relaxation could propose candidate coordinates, but the nonlinear dependence on unknown rates must not be described as a linear system. A conservative first implementation should keep the exact discrete rate grid used here and extend it only when measured misses justify the complexity.

### 5 A total eventual-sign worker with explicit witnesses

#### 5.1 Why eventual goals deserve their own certificate

A positivity certificate on  $[a, \infty)$  answers a different question from “there is some  $a$  beyond which the function is positive.” The second query should not be handled by guessing a large number until a bounded search happens to succeed. For (1), a direct exact algorithm constructs a cutoff for every nonzero canonical input.

This worker is particularly useful for existential goals. It returns a rational witness  $X$  together with a proof obligation that is uniform over every  $x \geq X$ . It can also supply the tail component of a proof whose bounded component is discharged by another worker. A tail receipt alone must never be silently promoted to a proof over the original entire domain.

## 5.2 An explicit domination theorem

Let  $f \neq 0$ , let  $\lambda_*$  be its largest rate, and let  $d$  be the degree of  $P_{\lambda_*}$ . Multiply the target by a sign  $\sigma \in \{-1, 1\}$  so that the leading coefficient of  $\sigma P_{\lambda_*}$  is  $a > 0$ . Define

$$\begin{aligned} L &= \sum_{k < d} |[x^k]P_{\lambda_*}|, \\ C_\lambda &= \sum_k |[x^k]P_\lambda|, \quad d_\lambda = \deg P_\lambda, \quad \delta_\lambda = \lambda_* - \lambda > 0, \\ k_\lambda &= \max(d_\lambda - d, 0) + 1, \\ B &= \sum_{\lambda < \lambda_*} \frac{C_\lambda k_\lambda!}{\delta_\lambda^{k_\lambda}}. \end{aligned}$$

Empty sums are zero. All these quantities are rational except for the integer degrees and orders.

**Theorem 5.1** (Exact eventual-sign certificate). *For*

$$X = \max\left(1, \frac{2L}{a}, \frac{4B}{a}\right), \quad (8)$$

one has

$$\sigma f(x) \geq \frac{a}{4} x^d e^{\lambda_* x} > 0 \quad \text{for every } x \geq X. \quad (9)$$

*Proof.* For  $x \geq 1$ , the lower-degree terms of the leading block have total magnitude at most  $Lx^{d-1}$ ; when  $d = 0$ , there are no such terms and  $L = 0$ . Hence

$$\sigma P_{\lambda_*}(x) \geq ax^d - Lx^{d-1} \geq \frac{a}{2}x^d$$

when  $x \geq 2L/a$ .

For a lower-rate block,  $|P_\lambda(x)| \leq C_\lambda x^{d_\lambda}$ . The exponential series, or the Taylor-remainder ladder of Section 3, gives  $e^t \geq t^k/k!$  for  $t \geq 0$  and  $k \in \mathbb{N}$ . Since  $\delta_\lambda x > 0$ ,

$$\begin{aligned} |P_\lambda(x)|e^{-\delta_\lambda x} &\leq \frac{C_\lambda k_\lambda!}{\delta_\lambda^{k_\lambda}} x^{d_\lambda - k_\lambda} \\ &\leq \frac{C_\lambda k_\lambda!}{\delta_\lambda^{k_\lambda}} x^{d-1}. \end{aligned}$$

The last step uses  $d_\lambda - k_\lambda \leq d - 1$  and  $x \geq 1$ . Summing yields a lower-rate contribution of magnitude at most  $Bx^{d-1} \leq (a/4)x^d$  when  $x \geq 4B/a$ . Factoring out  $e^{\lambda_* x} > 0$  now gives (9). Strictness follows from  $a > 0$  and  $x \geq 1$ .  $\square$

**Corollary 5.2** (Eventual sign and identity are decidable in the exact fragment). *The eventual sign of a nonzero canonical exponential polynomial is the sign of the leading coefficient in its largest-rate block. The zero canonical object is identically zero. Conversely, a nonzero canonical object cannot represent the zero function, by Theorem 5.1.*

This is a genuine completeness statement for the eventual-sign problem in the specified rational fragment, subject only to the practical limits of exact computation. It does not decide where the sign changes, whether the function is nonnegative on an earlier compact interval, or whether its minimum is zero. Those are different problems.

### 5.3 The search and the checker

The producer extracts the largest rate, leading degree and sign, computes the norms above, selects the displayed  $k_\lambda$ , and emits  $X$  with those orders. The checker independently requires

$$X \geq 1, \quad a > 0, \quad aX \geq 2L, \quad aX \geq 4B,$$

and verifies for every lower block that its supplied integer  $k_\lambda$  is positive and satisfies  $k_\lambda \geq d_\lambda - d + 1$ . It does not require the producer's exact order choice or its exact minimum cutoff. A different search can choose larger orders or a larger cutoff without changing the acceptance theorem.

The deliberately elementary cutoff may be quite conservative. For

$$f(x) = e^x - 1000x^5$$

one has  $a = 1$ ,  $L = 0$ ,  $k = 6$ ,  $B = 1000 \cdot 6! = 720000$ ; thus  $X = 2880000$  is an immediate certificate. This is not a useful numerical estimate of the last zero. It is a total, exact fallback that makes no root-isolation assumption.

A cheap refinement tries the native ladder at increasing anchors. On this example the tested sequence 0, 1, 2, 4, 8, 16, 32 first obtains a certificate at 32. That certificate proves strict positivity for all  $x \geq 32$  using exact rational enclosures of its initial values. The experiment establishes neither that 32 is the optimal cutoff nor that this anchor search always stays cheap. Its role is to improve a witness whose existence is already established by a different total algorithm.

### 5.4 Why sufficiently large native anchors work mathematically

There is a useful bridge between the two workers. Suppose the eventual leading sign is positive and use the native ascending factors. Every nonzero prefix function also has positive eventual leading sign. Before factors for the largest rate are reached, each lower factor  $(D - \rho)$  multiplies the leading coefficient of the largest-rate block by the positive quantity  $\lambda_* - \rho$ . Once the largest-rate factors are reached, all lower-rate blocks are already dead; the remaining factors differentiate its polynomial, preserving a positive leading coefficient until the function vanishes.

There are finitely many prefixes. Apply Theorem 5.1 to each nonzero one and choose an anchor larger than all resulting cutoffs. Every initial coordinate is then positive, and the native ladder succeeds. Thus a sufficiently large anchor exists without any auxiliary factors.

This argument distinguishes mathematical termination from the delivered bounded search. A fair search over unbounded anchors and increasing enclosure precision can eventually establish all the strictly positive initial values. The actual experiment tries a finite sequence at fixed precision, with the explicit tail certificate as its fallback. It makes no universal runtime claim.

### 5.5 Quantitative asymptotics available from the same proof

The estimates also give, for  $x \geq 1$ ,

$$|\sigma e^{-\lambda_* x} x^{-d} f(x) - a| \leq \frac{L + B}{x}. \quad (10)$$

Consequently, for any rational  $\eta > 0$ , the rational witness

$$X_\eta = \max \left( 1, \frac{L + B}{a\eta} \right)$$

ensures relative error at most  $\eta$  with respect to the leading term. The proof is the same triangle-inequality estimate, without splitting the budget into halves and quarters.

This corollary suggests an additional Forge contract:

```
leadingTermWithModulus(f, eta) ->
  (sign, rate, degree, coefficient, cutoff, proof)
```

Such a result can feed ratio comparisons, limits, and eventual denominator nonvanishing. The archive implements the eventual-sign contract, not this separate modulus API; the mathematical corollary is stated explicitly so that the next implementation does not need to discover another method.

If the source variable is restricted to naturals, replacing a rational cutoff by its ceiling gives an integer witness after a proved coercion step. If the source is a logarithmic chart  $x = e^t$ , a rational cutoff in  $t$  corresponds to the real witness  $e^X$  in  $x$ . The witness need not itself be rational. These transport steps belong to source reconstruction, not to rational rounding in the search process.

## 5.6 Combining a tail with a bounded-domain proof

For a requested ray  $[a, \infty)$  and a tail cutoff  $X \geq a$ , produce two separate subgoals:

$$\forall x \in [a, X], f(x) \geq 0, \quad \forall x \geq X, f(x) \geq 0.$$

The source proof uses the elementary dichotomy  $x \leq X$  or  $X \leq x$ . The shared endpoint must be covered; there is no half-open gap. If the target is strict, both component contracts must carry enough strict information.

This decomposition is especially attractive for a function that decreases initially and then becomes positive. It is not by itself a universal nonnegativity procedure: a compact component containing a zero can still defeat a simple interval worker. The derivative and factorisation routes are needed precisely at those contacts.

## 6 Exact initial values and bounded-domain coverage

### 6.1 Rational bounds for an exponential

The origin case is rational and should stay purely symbolic. For other rational anchors, a small enclosure procedure is sufficient. Let  $z \in \mathbb{Q}$  and choose  $s \geq 0$  so that  $r = |z|/2^s \leq 1/2$ . For an integer  $n \geq 1$ , set

$$S_n(r) = \sum_{k=0}^n \frac{r^k}{k!}, \quad T_n(r) = \frac{r^{n+1}}{(n+1)!} \frac{1}{1 - r/(n+2)}.$$

All denominators are strictly positive. The successive terms of the tail starting at degree  $n+1$  have ratios at most  $r/(n+2)$ , so

$$S_n(r) \leq e^r \leq S_n(r) + T_n(r). \tag{11}$$

For  $z < 0$ , invert this interval before the repeated squaring:

$$\frac{1}{S_n + T_n} \leq e^{-r} \leq \frac{1}{S_n}.$$

Squaring a nonnegative interval is monotone. Repeating it  $s$  times produces bounds on  $e^z$ .

The implementation rounds lower endpoints down and upper endpoints up to a fixed dyadic grid after arithmetic stages. If the denominator is  $2^b$ , the rounding operations are exact integer floor and ceiling operations, not floating-point conversions. Repeated squaring of a very small interval can therefore produce a lower endpoint zero; this loses precision but remains sound. It never converts an uncertain lower sign into a positive claim.

## 6.2 Normalising an entire point evaluation

Direct evaluation at a large positive anchor can create enormous intermediate numbers even when the sign is easy. First evaluate every polynomial block  $P_\lambda(a)$  exactly, deleting zero values, and choose

$$M = \max_{P_\lambda(a) \neq 0} (\lambda a).$$

Then

$$f(a) = e^M \sum_{\lambda} P_\lambda(a) e^{\lambda a - M}. \quad (12)$$

Every remaining exponential argument is nonpositive. The normalised sum has the same sign as the original value and avoids computing an enormous common positive scale. Its enclosure is what the receipt records.

This distinction matters when reading the JSON: the recorded lower and upper numbers at a nonzero anchor bound a *positively normalised* value, not necessarily  $f(a)$  itself. The checker recomputes that same mathematical normalisation. An integration layer which exports those numbers as bounds on  $f(a)$  without multiplying by the common factor would be wrong.

When the normalised coefficients cancel exactly before enclosure, the result is exact. When the enclosure straddles zero, the producer returns unknown at its current precision. The prototype does not use a decimal tolerance, a “nearly zero” test, or an oracle’s numerical sign as a proof.

## 6.3 A reference interval cover, not a new interval tactic

For experiments combining compact and unbounded domains, the archive includes a simple natural-interval worker. It evaluates polynomial blocks by interval Horner arithmetic, bounds each exponential using monotonicity and (11), sums the resulting intervals, and bisects when the lower bound is negative.

Each leaf records its rational endpoints and its recomputed enclosure. An internal node records a rational split point and two child nodes. The checker requires that the split lie strictly inside the parent’s interval and that the children cover exactly its left and right pieces. A collection of individually valid positive boxes is insufficient without this coverage proof.

Normalisation on an interval uses one common positive constant for the *whole leaf*. The chosen scale bounds every rate times every point in the leaf, so the remaining exponential arguments are nonpositive. Normalising each summand by a different factor would alter their relative magnitudes and would be unsound. The implementation comments call out this specific failure mode.

This cover is deliberately not presented as a contribution competing with Forge’s existing polynomial Bernstein worker or with Lean interval libraries. Its purpose is to exercise the new analytic interface, certify finite patches, and retain exact counterpoints. Production should reuse proved interval infrastructure rather than adopting a second unverified interval stack.

## 6.4 Why subdivision cannot replace the ladder

For  $f(x) = e^x - 1 - x$  on  $[0, h]$ , natural interval evaluation contains the range

$$[1, e^h] - 1 - [0, h] = [-h, e^h - 1].$$

Every  $h > 0$  gives a negative lower endpoint, even though  $f$  is nonnegative. Splitting the interval leaves a first piece containing zero, and the same obstruction persists. The problem is lost correlation between  $e^x$  and  $x$ , not insufficient effort on a remote part of the domain.

The differential ladder establishes the exact second-order contact at the endpoint with rational initial values. This comparison is not a claim that all sophisticated Taylor-model or symbolic interval methods must fail; a method which explicitly factors the contact or uses the same derivative information can succeed. It is a precise explanation of why this delivered natural-interval worker returns unknown while the ladder succeeds.

Conversely, interval subdivision certifies strictly positive patches of functions whose native ladder fails. In the experiment, the four instances of  $f_\varepsilon$  on  $[0, 2]$  use respectively 14, 21, 31, and 66 leaves as  $\varepsilon$  decreases through 1, 1/2, 1/4, 1/16. The proof obligations are bounded-domain statements, unlike the auxiliary ladders' whole-ray statements. Their counts must not be compared as though the workers solved the same goal.

## 6.5 Negative points are useful, but have a different type

A failed enclosure is not a counterexample. The worker records a rational counterpoint only when a point enclosure has *strictly negative upper bound*. For  $e^x - 2$  on  $[0, 1]$ , it finds such a point and the independent replay checks its membership in the original interval and the negative upper bound.

A counterpoint receipt answers a refutation request. It does not make a nonnegativity goal successful. Likewise a tail with negative eventual sign provides valuable information about an impossible requested positive tail, but its logical polarity must remain explicit. The JSON goal type and `verify_goal` enforce these distinctions.

# 7 From source expressions to Lean proofs

## 7.1 A small semantic bridge rather than an unverified translation

The first end-to-end target should be an origin-anchored inequality over a literal exponential polynomial. Its proof construction has four distinct objects:

$$\text{source expression} \xrightarrow{\text{proved equality}} \text{eval}(p) \xrightarrow{\text{checked ladder}} \text{NonnegativeRay}(p, a) \xrightarrow{\text{domain transport}} \text{original goal}.$$

The arrows represent proofs, not merely successful parsing or a matching hash. The final arrow is trivial only when the source and certificate domains and strictness already coincide.

A proposed Lean interface is:

```

-- Interface sketch; names are proposed, not existing Forge declarations.
structure ExpPoly where
  blocks : Array (Rat x Array Rat)

structure ReifiedExpPoly where
  source      : Expr
  polynomial  : ExpPoly
  evalEq      : Expr -- proof: source = eval polynomial at the chosen variable
  variable    : FVarId
  contextKey  : ContextKey

structure LadderReceipt where
  anchor      : Rat
  factors     : Array Rat
  pointCertificates : Array PointCertificate

checkLadder_sound :
  checkLadder p c = true ->
  forall x : Real, c.anchor <= x -> 0 <= eval p x

```

The syntax in this sketch uses readable placeholders such as  $x$  for a product and `Real` for the real type; it is not advertised as compiling Lean. The supplied `LadderBridge.lean`, in contrast, is actual candidate Lean source with the full intended theorem and proof, but remains uncompiled.

The internal polynomial representation should have a well-formedness predicate for ordered unique rates, trimmed coefficient arrays, and exact rationals. Either use a subtype carrying that proof, or canonicalise before every checked operation. Do not permit two distinct encodings of the same block to bypass a terminal-zero or source-equality check.

## 7.2 Logarithmic charts expand the useful source language

For  $x > 0$ , put  $t = \log x$ , so  $x = e^t$ . An expression

$$\sum_{j,k} c_{j,k} x^{r_j} (\log x)^k$$

with rational exponents becomes  $\sum_{j,k} c_{j,k} t^k e^{r_j t}$  once the real-power identities and their domain conditions have been proved. Integer negative powers also fit this pattern. The map is particularly convenient on  $x \geq 1$ , which corresponds to  $t \geq 0$ . On  $0 < x \leq 1$ , use  $u = -\log x \geq 0$  and reflection.

The chart is not permission to rewrite real powers at nonpositive arguments. A denominator must be proved positive or nonzero before clearing it; a square root must be connected to its exponential representation on the positive domain; and  $\log 1 = 0$  must preserve whether the original endpoint is included.

**Example 7.1** (Entropy inequality). For  $x > 0$ ,

$$x \log x - x + 1 \geq 0.$$

Its chart is  $f(t) = (t - 1)e^t + 1$ . On  $t \geq 0$ , native factors  $(0, 1, 1)$  have initial vector  $(0, 0, 1)$ . On  $t \leq 0$ , reflect:  $f(-u) = 1 - (u + 1)e^{-u}$ , whose factors  $(-1, -1, 0)$  also have initial vector  $(0, 0, 1)$ . Thus the source inequality follows on both sides of one, with equality at one and strict positivity elsewhere.

**Example 7.2** (A rational lower bound for the logarithm). For  $x \geq 1$ ,

$$\log x \geq \frac{2(x - 1)}{x + 1}.$$

After proving  $x + 1 > 0$  and writing  $x = e^t$ , the target is

$$(t - 2)e^t + t + 2 \geq 0 \quad (t \geq 0).$$

Factors  $(0, 0, 1, 1)$  have initial vector  $(0, 0, 0, 1)$ . The third-order boundary contact is handled exactly; no interval precision is spent resolving it.

**Example 7.3** (A square-root upper bound for the logarithm). For  $x \geq 1$ ,

$$\log x \leq \frac{x - 1}{\sqrt{x}}.$$

The proved positive-domain chart gives

$$e^{t/2} - e^{-t/2} - t \geq 0 \quad (t \geq 0).$$

Factors  $(-1/2, 0, 0, 1/2)$  have initial vector  $(0, 0, 0, 1/4)$ .

The archive records certificates for these *chart polynomials*. It does not contain an executed Lean reifier proving the original logarithmic source goals. The transformations above are mathematical derivations and concrete acceptance targets for that reifier; they are not additional kernel-checked experiment rows.

### 7.3 Composing the worker with existing algebra

Several useful extensions require no new analytic checker. For a quotient  $f/g$  with a proved nonzero denominator, its derivative has numerator  $f'g - fg'$  and denominator  $g^2 > 0$ . The numerator is again an exponential polynomial if  $f$  and  $g$  are. Certifying its sign establishes monotonicity; certifying the corresponding next derivative numerator can establish convexity. The denominator obligations must remain attached to the original domain.

A common positive factor may be removed. For example,

$$e^x \geq e^y(1 + x - y)$$

reduces, after multiplication by  $e^{-y} > 0$ , to  $e^t \geq 1 + t$  with  $t = x - y$ . A proof over all real  $t$  combines the two reflected ray certificates. This handles a multivariate source through a one-dimensional proved reduction; it is not a general multivariate exponential procedure.

Similarly, polynomial factors known to be nonnegative should be retained outside the ladder whenever possible. Expanding a square only to force an analytic search to rediscover positivity wastes both algebraic structure and proof size. The router should compare the expanded derivative representation with a factored source view rather than making either view globally canonical.

### 7.4 Concrete libraries to reuse

**Mathlib derivative and monotonicity theorems.** Use the explicit-derivative API in `Mathlib/Analysis/Calculus/Deriv/MeanValue.lean` and `Mathlib/Analysis/SpecialFunctions/ExpDeriv.lean`. These expose the monotonicity and exponential chain-rule ingredients for the step theorem [7, 8]. Explicit `HasDerivAt` witnesses avoid the default-zero behaviour of a totalised `deriv` when differentiability has not been established.

**LeanCert for bounded analytic obligations.** LeanCert’s maintained interface documents checked interval evaluation, automatic differentiation, bounds, optimisation, and root certificates. Programmatic entry points include `LeanCert.evalInterval`, `LeanCert.evalInterval_correct`, `LeanCert.API.Bounds`, and `LeanCert.API.Optimization`. Its documented automatic tail fragment is currently reciprocal-power bounds over naturals, rather than general exponential or logarithmic tails [10]. Thus it is a concrete candidate for the compact-domain and point-bound adapter, not a reason to reimplement its interval evaluator inside Forge.

Trust selection must be explicit. LeanCert documents a native mode using `native_decide`, a kernel mode, and an automatic mode that can fall back. For the strict kernel-only route envisioned here, request `leancert (trust := kernel)` or use the corresponding checked API without native fallback. This archive inspected those interfaces; it did not install, run, or benchmark LeanCert.

**IntervalBounds for small concrete endpoint estimates.** The `peti12352/lean-interval-bounds` project provides `approx`, `interval_decide`, and `norm_num` extensions for exact transcendental bounds. Its documented approach avoids floating-point proof arithmetic and `native_decide`. It is narrower than a quantified interval worker, but well matched to rational-anchor estimates. Its README reports a `v4.33.0-rc1` toolchain, different from the audited Forge baseline; compatibility must be compiled rather than assumed [9].

**Exact arithmetic and algebraic side conditions.** The prototype needs only Python’s `Fraction`, integer arithmetic, arrays, and maps. A first Lean implementation can use rational coefficients and proved sparse operations, plus ordinary ring normalisation and numerical proofs for small emitted identities. No SAT, SMT, SDP, or numerical eigenvalue library is required for the native ladder or the total tail constructor. Adding one would complicate deployment without solving a missing subproblem in these algorithms.

The library recommendation is therefore narrow: Mathlib for the analytic bridge, a proved exact polynomial representation for the certificate arithmetic, LeanCert or IntervalBounds for endpoint and compact obligations, and Forge’s existing algebraic worker for factors. These pieces have separate responsibilities and no implied claim of plug-and-play version compatibility.

## 7.5 Candidate proof emission in the archive

The supplied `LadderBridge.lean` states the one-step theorem in terms of arbitrary real functions and explicit derivative proofs. The emitter then creates a private definition for every  $q_j$ , proves its derivative identity, proves its initial value by exact arithmetic, and applies the step theorem backwards from the zero function. This is intentionally a proof-producing path, not an invocation of the Python verifier from Lean.

The four generated modules cover the Taylor remainder of order four, the positive entropy chart, the rational logarithm-bound chart, and the auxiliary-factor example with  $\varepsilon = 1$ . Their final statements say nonnegativity on the closed nonnegative ray. Some corresponding Python receipts establish stronger strict conclusions, but the emitted Lean statements are not presented as closing those stronger requests.

Every candidate requests an axiom inventory. None of those requests was executed because this environment has no Lean executable. The first integration milestone must compile and repair the bridge and all four replays without weakening their statements or introducing proof holes. Only then should proof emission be replaced with a reflected checker for larger certificates.

## 7.6 Why a reflected checker is the second step, not the first

A reflected checker should prove soundness once for evaluation, shifted differentiation, point certificates, and ladder recursion. It then reduces a closed Boolean certificate check and invokes the soundness theorem. This can avoid a large tower of repeated ring proofs, especially for auxiliary-factor certificates with long integer coefficients.

However, reflection introduces additional obligations: correctness of rational casts, polynomial evaluation, sorting and trimming, and the executable arithmetic backend. A small emitted proof is an excellent reference oracle while those lemmas are developed. Keep the source-emission path as a fallback and differential test even after reflection becomes the production route.

## 8 Executed experiments and what they establish

### 8.1 Environment, inputs, and receipts

The experiments ran with Python 3.13.5 on Linux x86-64, using the standard library only. Commands were invoked with `python -S`, which disables site-package initialisation. The seed for the recorded experiment bundle is 20260915. Full interpreter and platform strings are retained in `results/experiment-summary.json`.

The original canonical problems and requested goal types are stored in `results/problems.json`. The separate receipt stream in `results/certificates.json` references those problems by identifier. The replay program reopens both files and calls `verify_goal`; it does not trust a polynomial or domain supplied only by the receipt. All 284 stored receipts are accepted against their recorded requests.

| Family                           | Receipts | Actual certified request                                             |
|----------------------------------|----------|----------------------------------------------------------------------|
| Taylor boundary zeros            | 25       | Strict positivity on $x > 0$ , with equality at zero                 |
| Named exponential/chart examples | 8        | Strict positivity of the recorded chart polynomial on $x > 0$        |
| Auxiliary-factor examples        | 4        | Strict positivity on the closed nonnegative ray                      |
| Reference compact covers         | 4        | Nonnegativity on $[0, 2]$                                            |
| Random eventual signs            | 240      | Explicit eventual sign, positive or negative according to the source |
| Growth witnesses                 | 2        | One eventual-sign receipt; one strict ray receipt at anchor 32       |
| Negative point                   | 1        | A counterexample in the requested interval                           |
| Total stored receipts            | 284      | A count of receipts, not 284 Lean theorems                           |

This total is internal to one bundle. It is not added to the eighteen older Forge runs. Moreover, some receipts intentionally concern the same function with different domains or result types. They are not independent discoveries or a solved-goal percentage on a theorem corpus.

The eight named cases include the reflected exponential tangent remainder, quadratic and cubic hyperbolic remainders, both entropy charts, the two logarithmic bounds, and a damped positive square. The random eventual-sign generator creates canonical sums from eight random monomial proposals, rates with denominator three, degrees below six, and small signed integer coefficients. It tests the total tail constructor on varied exact inputs; it is not sampled evidence of the universal theorem, whose proof is in Section 5.

## 8.2 An ablation with an exhaustive finite oracle

The most informative algorithmic ablation tests the factor-order theorem. For each of 200 independently generated inputs with rates drawn from  $\{-2, 1, 3\}$  and degrees at most one per block, the experiment enumerates every distinct permutation of the native multiset. Across the cases it checks 14,409 orders. Forty-four inputs have an accepted ascending ladder, and in every case

$$\text{ascending ladder accepts} \iff \text{some enumerated order accepts.}$$

There are zero mismatches. The records retain the number of accepting orders per case, so the ablation is reproducible rather than merely an assertion that both code paths “usually agree.”

This checks a finite consequence of Theorem 4.1 against exhaustive search. It does not replace the proof and it does not establish that every positive input among the other 156 cases lacks some richer certificate. The counterpart in the unit tests uses two rates and a separate seed.

## 8.3 Independent calculations and mutation tests

The test suite contains 18 test methods. Several methods contain hundreds of cases; the method count is not the case count. Important checks are:

**Differentiation through three representations.** Two hundred random inputs are differentiated through six random factors. The producer’s termwise loop is compared against the checker’s coefficient stencil. Initial values are also compared with a direct jet oracle:

$$D^m(x^k e^{\lambda x})|_{x=0} = \begin{cases} \frac{m!}{(m-k)!} \lambda^{m-k}, & m \geq k, \\ 0, & m < k. \end{cases}$$

The oracle multiplies the scalar characteristic polynomials  $\prod_j (z - \rho_j)$  and applies their coefficients to these ordinary jets. It does not generate the same ladder by calling the producer again.

**Annihilation and tail checks.** One hundred random native annihilators are checked to kill their inputs. Three hundred additional random tail receipts and the zero input are replayed in the unit suite. These cases are separate from the 240 retained random tail records; their counts are not pooled into an inflated certificate count.

**Enclosure diagnostics.** Two hundred rational exponential arguments are compared against `Decimal.exp` at 160-digit precision. This is a high-precision diagnostic oracle, not a proof of the enclosure theorem. Precision-refinement cases, negative arguments, zero arguments, and resource-parameter validation are also exercised. The mathematical enclosure proof is (11) and directed rounding.

**Corruption and source substitution.** Tests change a differential factor, an initial lower bound, a polynomial coefficient, a terminal length, a source polynomial, an anchor, a sign, a cutoff, a tail order, an interval endpoint, and a split point. They also try floating-point and Boolean values where exact numbers are required. The specified corruptions are rejected. The tests expressly reject using an anchor-one proof at anchor zero, a tail as a whole-ray proof, and a negative point as a successful nonnegativity proof.

These tests are evidence of the tested invariants. They are not a security audit of a hostile-input service. Input-size, coefficient-bit-length, recursion, and allocation limits still need production engineering.

## 8.4 Unknown results retained deliberately

Two instructive misses are retained in the summary. The interior-zero case  $(e^x - 2)^2$  remains unknown under the bounded auxiliary-factor search; its stronger all-factor obstruction is proved in Theorem 3.4. The natural-interval attempt on  $e^x - 1 - x$  over  $[0, 1]$  remains unknown at subdivision depth 14; the ladder certifies the same nonnegativity statement directly.

These misses are not hidden by counting only returned receipts. Conversely, an unknown result is never recorded as falsity of the target. A failed local certificate search and a checked negative point are different outcomes in both the experiment harness and the proposed worker API.

## 8.5 Replay separation and its limits

The replay process checks that the search module is not imported. It verifies stored receipts without regenerating them. Derivative replay uses a distinct calculation, and tail replay checks sufficient inequalities rather than recomputing the proposed cutoff.

There is an important shared component: point and interval enclosure arithmetic is used by both the corresponding searches and replay. A common error there could survive producer/checker agreement. The independent high-precision checks and elementary mathematical proof reduce that risk but do not eliminate it. A Lean soundness theorem for that component, or replacement by a proved library adapter, is the intended resolution.

The stored certificate JSON contributes 154,227 bytes when each individual receipt is serialised compactly and their sizes are summed. This excludes problem data, pretty-printing whitespace, indices, and other files. The largest single receipt in that measure is 14,694 bytes. These figures characterise the reference format, not compiled Lean proof sizes or memory consumption.

A single elapsed time is retained as environment metadata, but no timing claim is made against another prover or another backend. No Lean compilation, `grind` comparison, or corpus success-rate evaluation was performed in this environment.

# 9 An implementation sequence for Forge

## 9.1 The first useful end-to-end slice

Implement an origin-anchored native ladder before auxiliary factors, interval subdivision, or new user syntax. This slice has no transcendental numerical initial values: its boundary conditions are rational. The only analytic theorem needed is the integrating-factor step, and the only source language needed is literal real exponential polynomials.

A proposed source layout is:

|                                 |                                              |
|---------------------------------|----------------------------------------------|
| Forge/Analytic/ExpPoly.lean     | -- exact representation and evaluation       |
| Forge/Analytic/Deriv.lean       | -- evaluation of shifted differentiation     |
| Forge/Analytic/Ladder.lean      | -- analytic step and ladder soundness        |
| Forge/Analytic/Reify.lean       | -- source expression plus equality proof     |
| Forge/Analytic/Search.lean      | -- native and auxiliary-factor producers     |
| Forge/Analytic/PointBounds.lean | -- checked library adapter, not a new solver |
| Forge/Analytic/Tail.lean        | -- explicit domination certificate           |
| Forge/Analytic/Charts.lean      | -- reflection, log chart, positive factors   |
| Forge/Analytic/Worker.lean      | -- Forge obligation/result adapter           |
| Forge/Analytic/Tests.lean       | -- source goals, failures, strictness        |

These are proposed additions, not existing repository paths. The meaningful milestone is a theorem in an independent file, compiled from the original expression, whose proof can be replayed without the search process. A test that only applies the generic ladder theorem to assumed derivative equalities does not meet that milestone; it bypasses the source bridge that must be exercised.

## 9.2 Milestones with exact acceptance conditions

**M1: compile the analytic bridge and reference replays.** Compile `LadderBridge.lean` and all four emitted modules against a fixed Lean/Mathlib pair. Retain compiler output and axiom inventories. Every stated theorem must remain unchanged. This establishes the analytic proof skeleton, not an automatic tactic.

**M2: certify the sparse algebra and reifier.** Prove evaluation correctness for addition, multiplication, reflection, trimming, and (6). Reify original Lean expressions containing rational casts and expanded products. Require proofs of exact source equality; include intentionally permuted variables and differently scoped local identifiers in negative tests. A theorem about a manually entered canonical polynomial does not close the reifier milestone.

**M3: close native origin-anchored source goals automatically.** Generate the ascending spectrum, build a receipt, replay it, and reconstruct nonnegative or strict source conclusions. Add a minimal worker registration to Forge’s eventual controller, or a separate experimental tactic while that controller is unavailable. Do not claim an integrated `forge` command until one actually exists. Test both reflected rays and exact high-order boundary zeros.

**M4: add bounded auxiliary-factor search.** Reuse the same checker and record the first failed coordinate on each attempted multiset. Accept the four enrichment fixtures and retain the interior-zero control as unknown. Also run the existing algebraic worker on those fixtures: the goal is an improved portfolio, not forced use of the new analytic lane.

**M5: add checked endpoints and explicit tails.** Compile a point-bound adapter, choosing a kernel-only proof path explicitly. Prove Theorem 5.1 for the representation and emit its rational witness. Demonstrate that the tail of  $e^x - 1000x^5$  is accepted while a request for positivity starting at zero is not closed by that receipt. Then add the bounded-plus-tail gluing lemma.

**M6: add proved source charts and evaluate the portfolio.** Close the original entropy and logarithm source theorems, not merely their chart polynomials. Evaluate on held-out expressions and user developments,

including elaboration time, search time, replay time, proof size, and axiom inventory. Preserve baseline imports, hypotheses, and resource limits across compared tactics.

### 9.3 Routing should exploit the theorems

A deterministic initial policy can be expressed without designing another whole-system scheduler:

```
solveAnalytic(obligation):
  inspect the exact source type, relation, and domain
  try a proved positive-factor or one-dimensional chart reduction
  if outside the supported fragment: return NotApplicable
  if an algebraic factorisation already proves the goal: use it
  if the requested anchor is zero: try the native ascending ladder
  if unsuccessful and the budget permits: try auxiliary multisets
  if the request is eventual: construct the exact tail certificate
  if the domain is compact: ask the checked interval/bounds adapter
  if a ray can be split: combine a compact receipt with a tail receipt
  return Unknown with the first unresolved obligation and diagnostics
```

The core mathematical routing rules are sharper than this pseudocode's order. Never search permutations after an exact ascending failure. Do not expect an auxiliary budget to repair an interior zero. Do not endlessly subdivide a boundary contact that has a cheap exact derivative ladder. Do not invoke a root solver merely to establish nonnegativity of a visible square.

The first failed ladder coordinate is diagnostic data, not necessarily a counterexample to the original inequality. A negative  $q_j(a)$  for  $j > 0$  can coexist with  $f > 0$  everywhere. The user-facing trace should identify it as a certificate obstruction and show the transformed function that failed.

### 9.4 Proof-cost controls

Rational arithmetic can grow rapidly with auxiliary multiplicity. Enforce limits on the number of rates, largest polynomial degree, number of factors, and total coefficient bit length. Charge the budget to both synthesis and reconstruction, so that a cheap search cannot return a receipt whose proof exceeds the entire tactic budget.

Keep exact caches keyed by the source polynomial, anchor, factor prefix, precision settings, and proof environment. A cache entry for a positively normalised point sign must not be misused as an unnormalised numerical bound. At the Lean level, include local context and universe identity in cache keys; the prototype's first-order JSON identifiers are not a substitute for those runtime identities.

Native ladders are small enough for proof emission. Longer ladders favour reflected checking and shared coefficient arrays. One can retain only factors and point receipts in the on-disk certificate and recompute intermediate polynomials in the checker. The independent reference emitter should remain available for differential tests of the reflected path.

### 9.5 A benchmark plan that would support real capability claims

Partition test families by their mathematical difficulty: boundary contact, non-native positive coordinates, interior zeros, eventual domination, source charts, and denominator side conditions. Include both factored and expanded presentations of the same functions. Randomly rename variables and reorder commutative source expressions to prevent success from depending on one surface spelling.

Compare the same original Lean goal against the baseline tactic configuration, the baseline plus the new worker, and the new worker with native-only, auxiliary-only, tail-only, and interval-adaptor ablations. Existing library lemmas must be available equally to both sides. Do not include the target theorem itself in a retrieval or simplification set for either side.

Report success only after the final source statement compiles and its proof meets the selected trust policy. Separate theorem counts from certificate counts, and separate a counterexample result from a solved positive goal. Measure kernel-only replay separately from search and from any native certificate evaluation. The current Python bundle provides useful fixtures for this experiment, not results of that experiment.

## 9.6 What would constitute a significant gain

The gain is not that Forge can rediscover  $e^x \geq 1 + x$  once more. It is that expanded analytic expressions can be routed to a finite exact certificate, that a false start in one coordinate system can be repaired by an explicitly stronger one, and that eventual statements acquire exact witnesses through a total constructor. These mechanisms produce information that the existing polynomial and discrete workers can consume without becoming analytic solvers themselves.

The most valuable first implementation is therefore compact: a proved source bridge, a native ascending ladder, and one checked analytic step. The auxiliary-factor and tail workers extend that same trusted core rather than requiring a new prover at each stage. The archive supplies enough executable search and replay evidence to make those stages concrete, while leaving the unperformed Lean integration clearly visible.

## A Certificate formats and API contracts

### A.1 Canonical source objects

A polynomial is a sorted array of triples `[rate, degree, coefficient]`. Rates and coefficients are strings in the canonical syntax of Python `Fraction`: an integer or a reduced fraction with a positive denominator. Examples are `"-2"`, `"1/2"`, and `"0"`. Inputs such as `"2/2"`, `"01"`, floats, and Boolean values are rejected by the decoder; a Boolean must not pass merely because Python treats it as an integer subtype. Zero coefficients, repeated keys, and unsorted rows are rejected at the encoded boundary.

For example,  $e^x - 1 - x$  has the source polynomial

```
[["0", 0, "-1"], ["0", 1, "-1"], ["1", 0, "1"]]
```

and a source request can be

```
{
  "polynomial": [["0",0,"-1"], ["0",1,"-1"], ["1",0,"1"]],
  "goal": {"kind": "positive_open_ray", "anchor": "0"}
}
```

These JSON objects are the prototype's independently supplied source problems. They are not a serialization of arbitrary Lean expressions. A Lean implementation must retain the separate source-evaluation equality discussed in Section 7.

## A.2 Ladder receipts

```
{
  "kind": "ladder-v1",
  "anchor": "0",
  "bits": 96,
  "order": 32,
  "steps": [
    {
      "rho": "0",
      "polynomial": [["0",0,"-1"], ["0",1,"-1"], ["1",0,"1"]],
      "lower": "0", "upper": "0"
    },
    {
      "rho": "0",
      "polynomial": [["0",0,"-1"], ["1",0,"1"]],
      "lower": "0", "upper": "0"
    },
    {
      "rho": "1", "polynomial": [["1",0,"1"]],
      "lower": "1", "upper": "1"
    }
  ]
}
```

The terminal zero is implicit and must be computed after the final factor. The precision parameters select a reproducible enclosure calculation, not a numerical trust tolerance. At a nonzero anchor the stored endpoints concern the positively normalised value of equation (12).

## A.3 Tail receipts

```
{
  "kind": "tail-v1",
  "sign": 1,
  "cutoff": "28800000",
  "orders": [["0", 6]]
}
```

For the separately supplied input  $e^x - 1000x^5$ , this is the certificate from Theorem 5.1. The checker extracts the leading rate, degree, coefficient, and all block norms from the source; it does not accept those facts merely because the receipt states them. The zero source admits a separate sign-zero receipt with no lower-rate orders.

## A.4 The accepted source request types

| Request                              | Required receipt conclusion                                             |
|--------------------------------------|-------------------------------------------------------------------------|
| <code>nonnegative_ray</code>         | A ladder with the identical anchor                                      |
| <code>positive_open_ray</code>       | Identical anchor and at least one strictly positive initial lower bound |
| <code>positive_closed_ray</code>     | Identical anchor and strictly positive first initial lower bound        |
| <code>eventual_sign</code>           | A tail receipt with the requested sign; cutoff is an explicit witness   |
| <code>nonnegative_interval</code>    | A complete cover of exactly the requested interval                      |
| <code>counterexample_interval</code> | A certified negative point in exactly that interval                     |

The public functions `verify(sourcePolynomial, receipt)` and `verify_goal(sourceProblem, receipt)` answer different questions. The former establishes the receipt's own conclusion against an independent polynomial; the latter also checks that this conclusion answers the actual request. Code invoking only the first function has not implemented the complete source-goal contract.

## B Reproduction and archive map

### B.1 Run the executable evidence

No third-party Python packages are needed. From the archive's `prototype` directory:

```
python -S -m unittest discover -s tests -v
python -S replay.py
python -S run_experiments.py
python -S emit_lean.py
```

By default, the experiment generator writes to `reproduced-results` rather than overwriting the retained results. The replay command defaults to the retained results. The original generation command explicitly selected the recorded output directory. Source, receipt, index, and order-ablation data are deterministic for the recorded seed; elapsed time and environment metadata are not expected to be identical across machines.

A small API example constructs and checks a Taylor certificate:

```
from forge_analytic.core import taylor_remainder, encode
from forge_analytic.search import ladder
from forge_analytic.check import verify_goal

p = taylor_remainder(8)
request = {
    "polynomial": encode(p),
    "goal": {"kind": "positive_open_ray", "anchor": "0"},
}
receipt = ladder(p)
assert receipt is not None
assert verify_goal(request, receipt)
```

The same interface can be used with `make`, `ghost_search`, `tail`, and `compact_cover`; their docstrings state the exact input and result conventions. These are algorithmic prototypes, not a call to a Lean kernel through Python.

## B.2 Build the article and inspect the candidates

The article uses ordinary pdfLaTeX and stock TeX packages:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge-analytic-certificates.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-analytic-certificates.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-analytic-certificates.tex
```

The Lean directory includes its own status and intended compilation commands. It does not include a vendored Mathlib checkout or a claim that an unchecked candidate is a finished proof. The baseline reported by the audited Forge snapshot is Lean v4.34.0 and Mathlib 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa [1].

## B.3 Files worth opening first

| Path                            | Purpose                                                              |
|---------------------------------|----------------------------------------------------------------------|
| article/                        | Editable article, section sources, references, and rendered PDF      |
| prototype/forge_analytic/       | Exact algebra, search, enclosures, and separate replay checks        |
| prototype/tests/                | Unit tests, independent jet oracle, and corruption controls          |
| results/experiment-summary.json | Counts, ablations, unknown controls, and environment                 |
| results/problems.json           | Independently supplied canonical source requests                     |
| results/certificates.json       | Recorded receipts linked to those requests                           |
| lean/                           | Candidate analytic bridge, four emitted replays, and explicit status |
| docs/                           | Repository audit, scope, and capability ledger                       |

No original Forge proposal, third-party implementation, binary toolchain, or font file is redistributed. The archive is an independent extension package; it does not modify the user's repository.

## References

- [1] V. Reshetnikov, *Forge: README*, snapshot c98e47c5f804e92880fc1d0e378c1b95832b685c, inspected September 15, 2026. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/README.md>.
- [2] V. Reshetnikov, *Forge implementation status*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/STATUS.md>.
- [3] Forge contributors, *Nonlinear arithmetic: exact certificates, not floating-point proofs*, merged article, Section 7, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/07-nonlinear.tex>.
- [4] Forge contributors, *Finite certificates for unbounded behaviour*, merged article, Section 6, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/06-closure.tex>.
- [5] V. Reshetnikov, *Leant: indexing composition diagnostics*, snapshot 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/reports/2026-09-14-indexing-composition-diagnostics.md>.
- [6] V. Reshetnikov, *Djex: indexing composition diagnostics*, snapshot e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/docs/reports/2026-09-14-indexing-composition-diagnostics.md>.
- [7] Mathlib contributors, *Mathlib.Analysis.Calculus.Deriv.MeanValue*, official API documentation, inspected September 15, 2026. [https://leanprover-community.github.io/mathlib4\\_docs/Mathlib/Analysis/Calculus/Deriv/MeanValue.html](https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/Calculus/Deriv/MeanValue.html). The moving documentation is an API reference, not a substitute for compiling the pinned project.
- [8] Mathlib contributors, *Mathlib.Analysis.SpecialFunctions.ExpDeriv*, official API documentation, inspected September 15, 2026. [https://leanprover-community.github.io/mathlib4\\_docs/Mathlib/Analysis/SpecialFunctions/ExpDeriv.html](https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/SpecialFunctions/ExpDeriv.html).
- [9] *IntervalBounds*, official repository README, inspected September 15, 2026; README blob 8c6b01fafe0e511d194f60a4997432eb7f7134c4. <https://github.com/peti12352/lean-interval-bounds>. Documented kernel-checked point-bound tactics and their supported domains.
- [10] *LeanCert*, official repository README, inspected September 15, 2026; README blob 8a65c45afc1763a36d5d406692779daff401b92f. <https://github.com/alerad/leancert>. Documented checked numerical APIs, trust modes, and current eventual-bound fragment.
- [11] A. Platzer and Y. K. Tan, *Differential Equation Invariance Axiomatization*, arXiv:1905.13429, 2019; Journal of the ACM 67(1), Article 6, 2020. <https://arxiv.org/abs/1905.13429>. Related general differential-invariant proof theory; not the implementation used here.
- [12] J. M. Aldaz, O. Kounchev, and H. Render, *Bernstein Operators for Extended Chebyshev Systems*, arXiv:0805.1612, 2008. <https://arxiv.org/abs/0805.1612>. Broader context for positive bases and the function spaces involved here.